35-如何监控Tomcat的性能？

专栏上一期我们分析了JVM GC的基本原理以及监控和分析工具，今天我们接着来聊如何监控Tomcat的各种指标，因为只有我们掌握了这些指标和信息，才能对Tomcat内部发生的事情一目了然，让我们明白系统的瓶颈在哪里，进而做出调优的决策。

在今天的文章里，我们首先来看看到底都需要监控Tomcat哪些关键指标，接着来具体学习如何通过JConsole来监控它们。如果系统没有暴露JMX接口，我们还可以通过命令行来查看Tomcat的性能指标。

Web应用的响应时间是我们关注的一个重点，最后我们通过一个实战案例，来看看Web应用的下游服务响应时间比较长的情况下，Tomcat的各项指标是什么样子的。

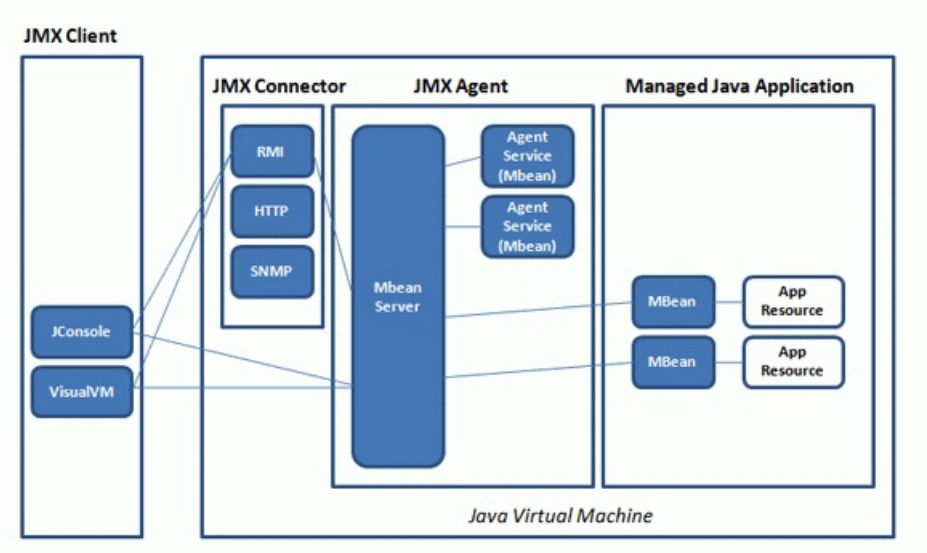
Tomcat的关键指标

Tomcat的关键指标有**吞吐量、响应时间、错误数、线程池、CPU以及JVM内存**。

我来简单介绍一下这些指标背后的意义。其中前三个指标是我们最关心的业务指标，Tomcat作为服务器，就是要能够又快有好地处理请求，因此吞吐量要大、响应时间要短，并且错误数要少。

而后面三个指标是跟系统资源有关的，当某个资源出现瓶颈就会影响前面的业务指标，比如线程池中的线程数量不足会影响吞吐量和响应时间；但是线程数太多会耗费大量CPU，也会影响吞吐量；当内存不足时会触发频繁地GC，耗费CPU，最后也会反映到业务指标上来。

那如何监控这些指标呢？Tomcat可以通过JMX将上述指标暴露出来的。JMX（Java Management Extensions，即Java管理扩展）是一个为应用程序、设备、系统等植入监控管理功能的框架。JMX使用管理MBean来监控业务资源，这些MBean在JMX MBean服务器上注册，代表JVM中运行的应用程序或服务。每个MBean都有一个属性列表。JMX客户端可以连接到MBean Server来读写MBean的属性值。你可以通过下面这张图来理解一下JMX的工作原理：



Tomcat定义了一系列MBean来对外暴露系统状态，接下来我们来看看如何通过JConsole来监控这些指标。

通过JConsole监控Tomcat

首先我们需要开启JMX的远程监听端口，具体来说就是设置若干JVM参数。我们可以在Tomcat的bin目录下

www.ixuexi.cf 分享站 用户标记 mannix小童鞋

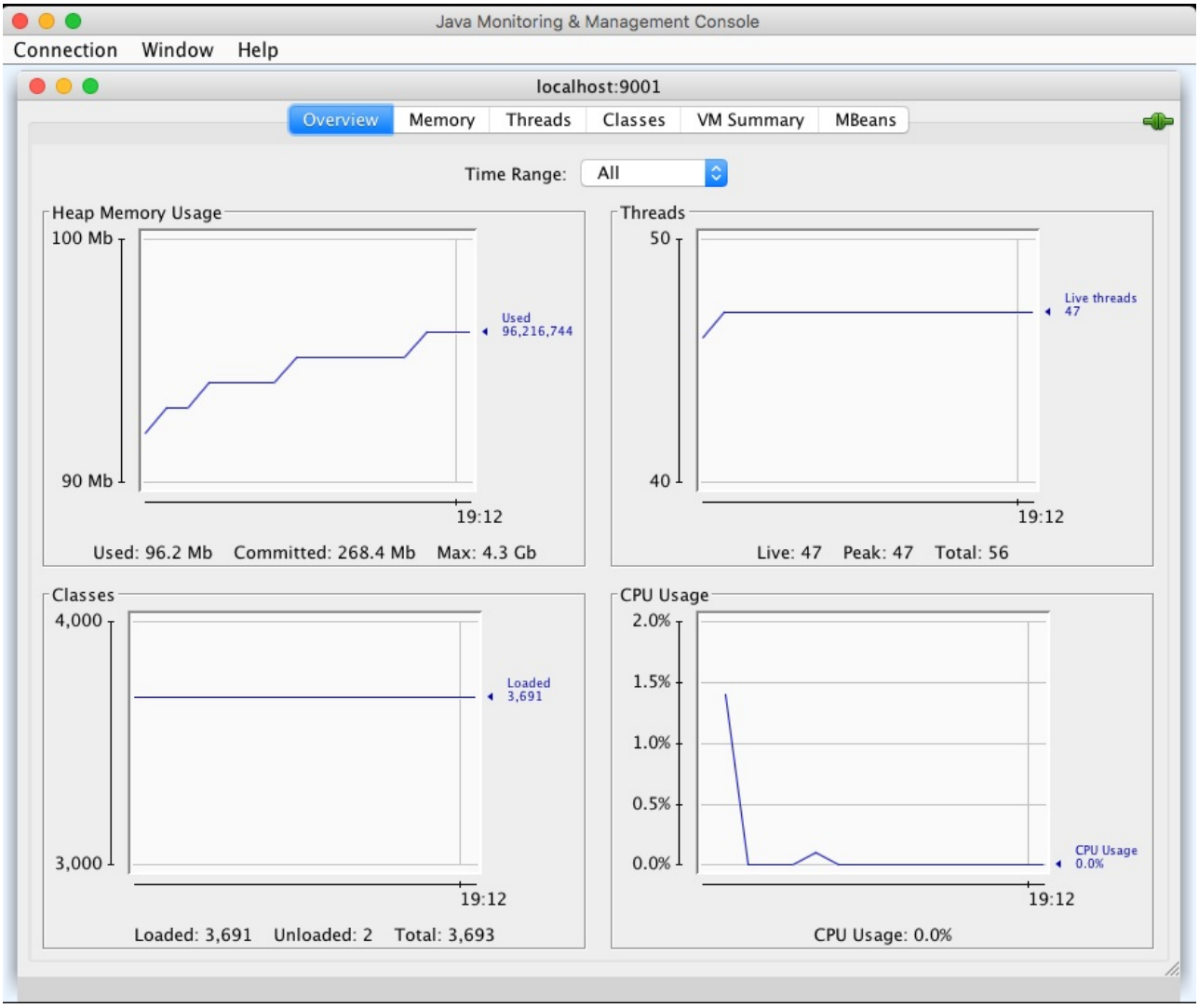
新建一个名为setenv.sh的文件（或者setenv.bat，根据你的操作系统类型），然后输入下面的内容：

```
export JAVA_OPTS="${JAVA_OPTS} -Dcom.sun.management.jmxremote"
export JAVA_OPTS="${JAVA_OPTS} -Dcom.sun.management.jmxremote.port=9001"
export JAVA_OPTS="${JAVA_OPTS} -Djava.rmi.server.hostname=x.x.x.x"
export JAVA_OPTS="${JAVA_OPTS} -Dcom.sun.management.jmxremote.ssl=false"
export JAVA_OPTS="${JAVA_OPTS} -Dcom.sun.management.jmxremote.authenticate=false"
```

重启Tomcat，这样JMX的监听端口9001就开启了，接下来通过JConsole来连接这个端口。

```
jconsole x.x.x.x:9001
```

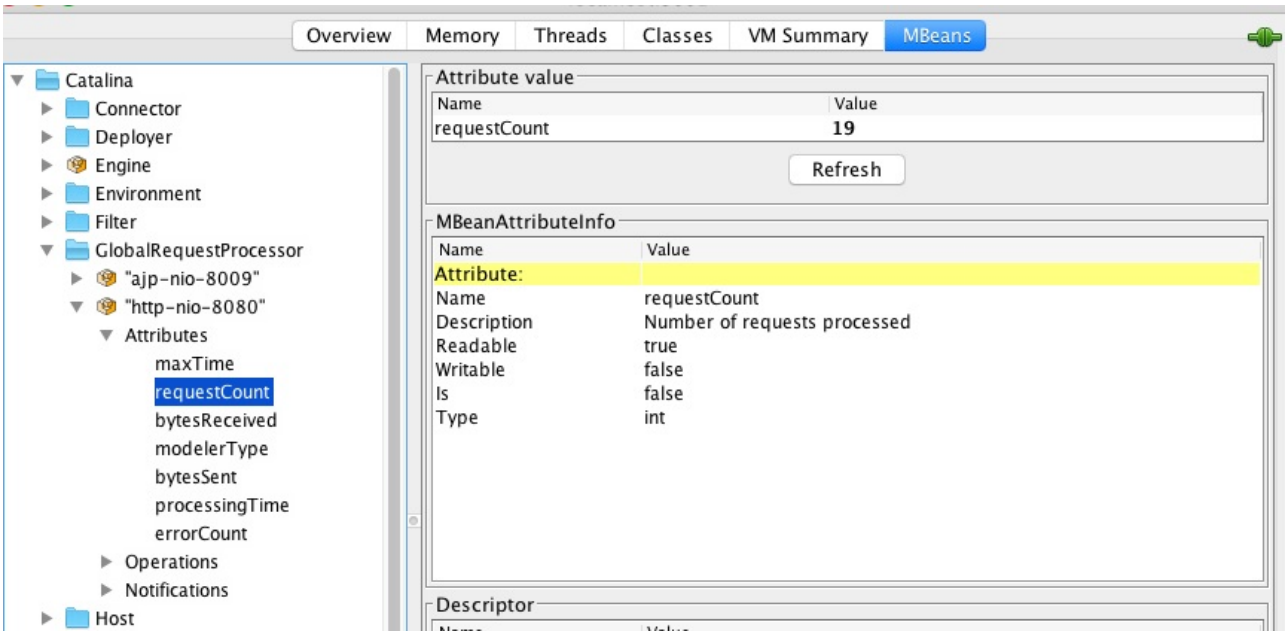
我们可以看到JConsole的主界面：



前面我提到的需要监控的关键指标有**吞吐量、响应时间、错误数、线程池、CPU以及JVM内存**，接下来我们就来看看怎么在JConsole上找到这些指标。

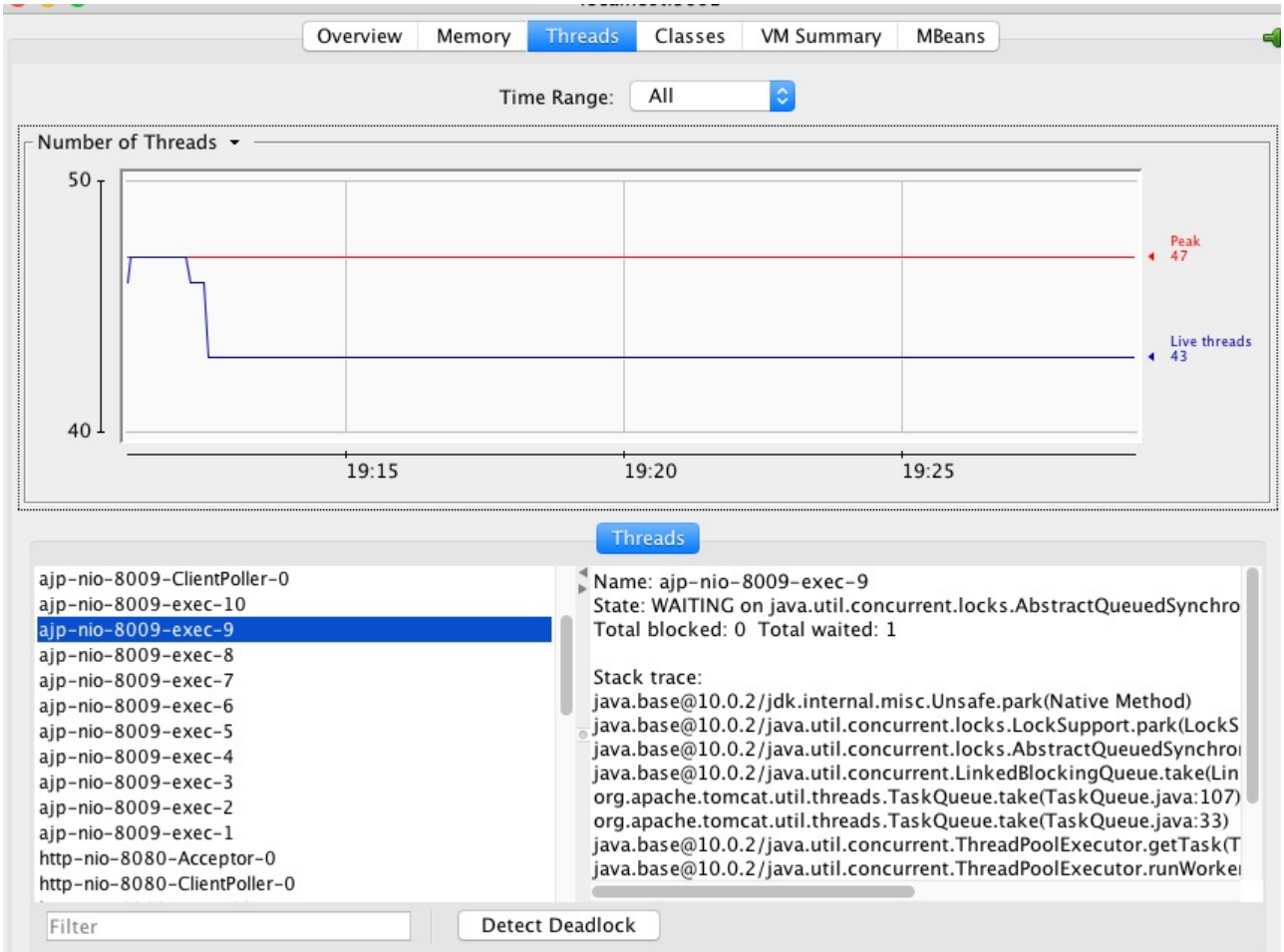
吞吐量、响应时间、错误数

在MBeans标签页下选择GlobalRequestProcessor，这里有Tomcat请求处理的统计信息。你会看到Tomcat中的各种连接器，展开“http-nio-8080”，你会看到这个连接器上的统计信息，其中maxTime表示最长的响应时间，processingTime表示平均响应时间，requestCount表示吞吐量，errorCount就是错误数。



线程池

选择“线程”标签页，可以看到当前Tomcat进程中有多少线程，如下图所示：

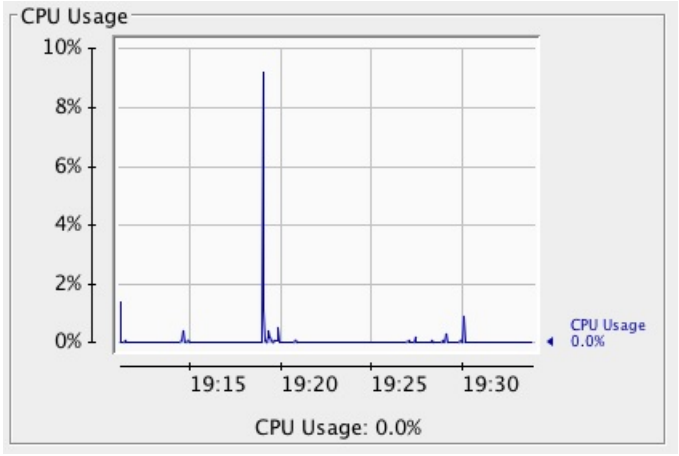


图的左下方是线程列表，右边是线程的运行栈，这些都是非常有用的信息。如果大量线程阻塞，通过观察线程栈，能看到线程阻塞在哪个函数，有可能是I/O等待，或者是死锁。

www.ixuexi.cf 分享站 用户标记 mannix小童鞋

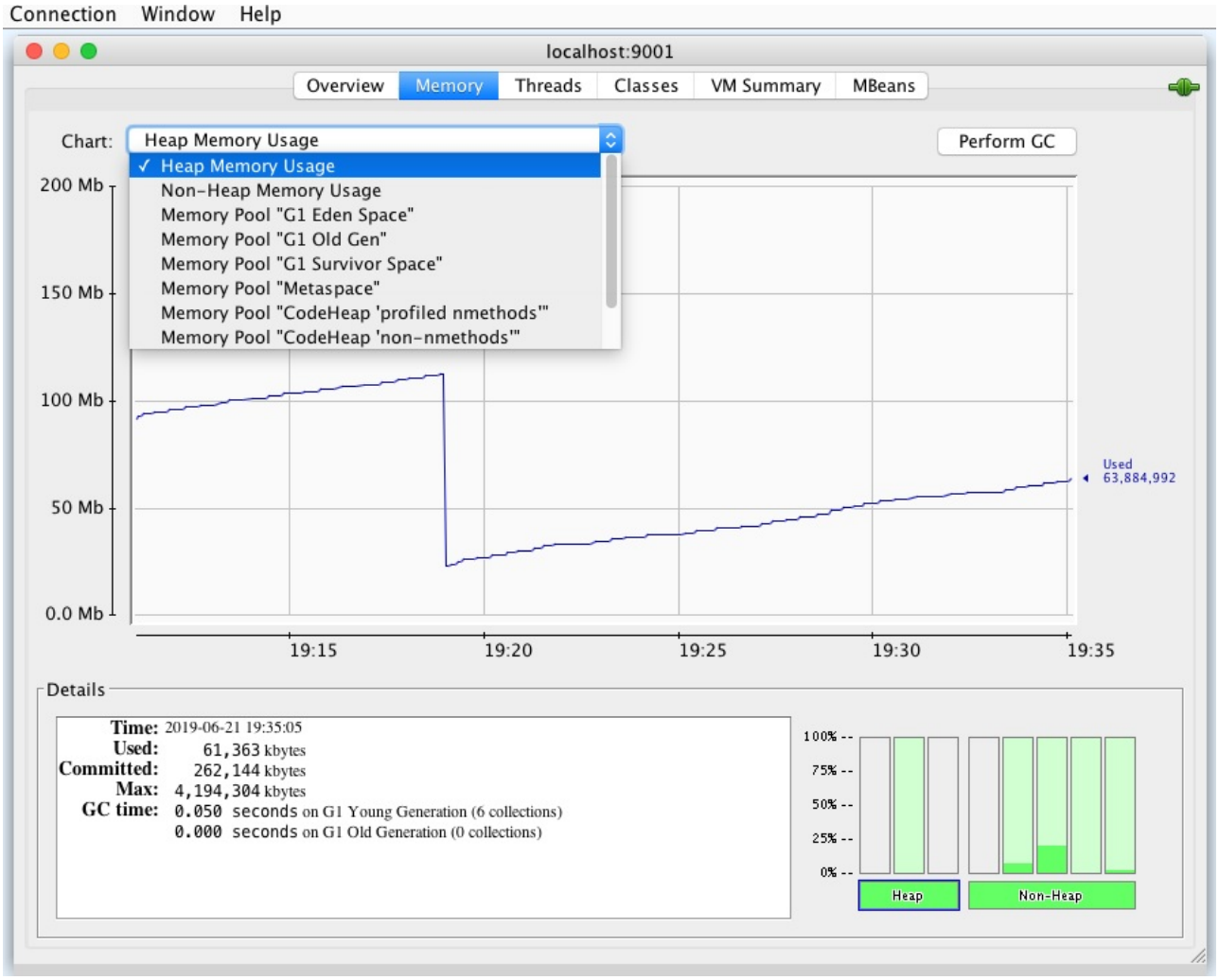
CPU

在主界面可以找到CPU使用率指标，请注意这里的CPU使用率指的是Tomcat进程占用的CPU，不是主机总的CPU使用率。

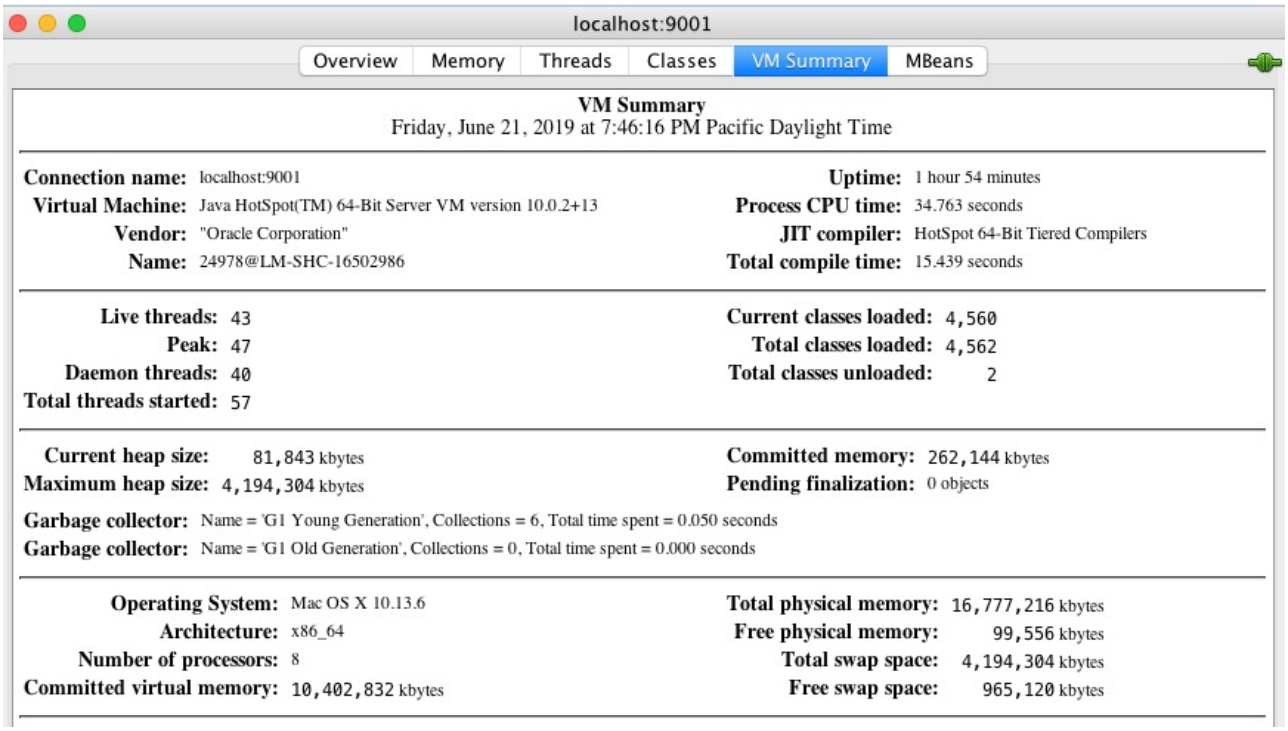


JVM内存

选择“内存”标签页，你能看到Tomcat进程的JVM内存使用情况。



你还可以查看JVM各内存区域的使用情况，大的层面分堆区和非堆区。堆区里有分为Eden、Survivor和Old。选择“VM Summary”标签，可以看到虚拟机内的详细信息。



命令行查看Tomcat指标

极端情况下如果Web应用占用过多CPU或者内存，又或者程序中发生了死锁，导致Web应用对外没有响应，监控系统上看不到数据，这个时候需要我们登陆到目标机器，通过命令行来查看各种指标。

1.首先我们通过ps命令找到Tomcat进程，拿到进程ID。

```
$ ps -ef | grep tomcat
```

2.接着查看进程状态的大致信息，通过cat/proc/<pid>/status命令：

www.ixuexi.cf 分享站 用户标记 mannix小童鞋

```
$ cat /proc/30943/status
Name:   java
State:  S (sleeping)
Tgid:   30943
Ngid:   0
Pid:    30943
PPid:   30940
TracerPid:      0
Uid:    78402   78402   78402   78402
Gid:    61000   61000   61000   61000
FDSize: 8192
Groups: 4 61000
NStgid: 30943
NSpid:  30943
NSpgid: 30939
NSsid:  30939
VmPeak: 16113056 kB
VmSize: 16092192 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:   8002064 kB
VmRSS:   7974888 kB
VmData: 16026468 kB
VmStk:       136 kB
VmExe:         4 kB
VmLib:     16916 kB
VmPTE:     26696 kB
VmPMD:        72 kB
VmSwap:        0 kB
HugetlbPages:          0 kB
Threads:        5873
SigQ:   0/48003
```

3.监控进程的CPU和内存资源使用情况：

```
$ top -p 30943
top - 21:10:23 up 59 days,  2:24,  1 user,  load average: 0.14, 0.28, 0.30
Tasks:   1 total,   0 running,   1 sleeping,   0 stopped,   0 zombie
%Cpu(s):  3.9 us,  0.4 sy,  0.0 ni, 95.6 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
KiB Mem:  12303860 total, 11363072 used,   940788 free,   132292 buffers
KiB Swap:        0 total,        0 used,        0 free.  2287284 cached Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
30943 cronusa+  20   0 15.347g 7.605g  14816 S  17.0 64.8   2071:47 java
```

4.查看Tomcat的网络连接，比如Tomcat在8080端口上监听连接请求，通过下面的命令查看连接列表：

```
$ netstat -na |  grep 8080
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.1:8080          127.0.0.1:44420         TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.102.153.28:40590     TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.173.117.164:26385    TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.173.117.164:10129    ESTABLISHED
tcp        0      0 10.199.1.103:8080       10.199.73.250:17460     TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.199.73.250:11936     TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.199.105.232:44204    TIME_WAIT
tcp        0      0 10.199.1.103:8080       10.102.153.25:41058     TIME_WAIT
```

你还可以分别统计处在“已连接”状态和“TIME_WAIT”状态的连接数：

```
$ netstat -na | grep ESTAB | grep 8080 | wc -l
12
$ netstat -na | grep TIME_WAIT | grep 8080 | wc -l
43
```

5.通过ifstat来查看网络流量，大致可以看出Tomcat当前的请求数和负载状况。

```
$ ifstat
       eth0
 KB/s in  KB/s out
   34.19    105.27
   37.62    143.22
   36.38    122.61
   35.40    115.54
```

实战案例

在这个实战案例中，我们会创建一个Web应用，根据传入的参数latency来休眠相应的秒数，目的是模拟当前的Web应用在访问下游服务时遇到的延迟。然后用JMeter来压测这个服务，通过JConsole来观察Tomcat的各项指标，分析和定位问题。

主要的步骤有：

1.创建一个Spring Boot程序，加入下面代码所示的一个RestController：

```
@RestController
public class DownStreamLatency {

    @RequestMapping("/greeting/latency/{seconds}")
    public Greeting greeting(@PathVariable long seconds) {

        try {
            Thread.sleep(seconds * 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        Greeting greeting = new Greeting("Hello World!");
```

www.ixuexi.cf 分享站 用户标记 mannix小童鞋

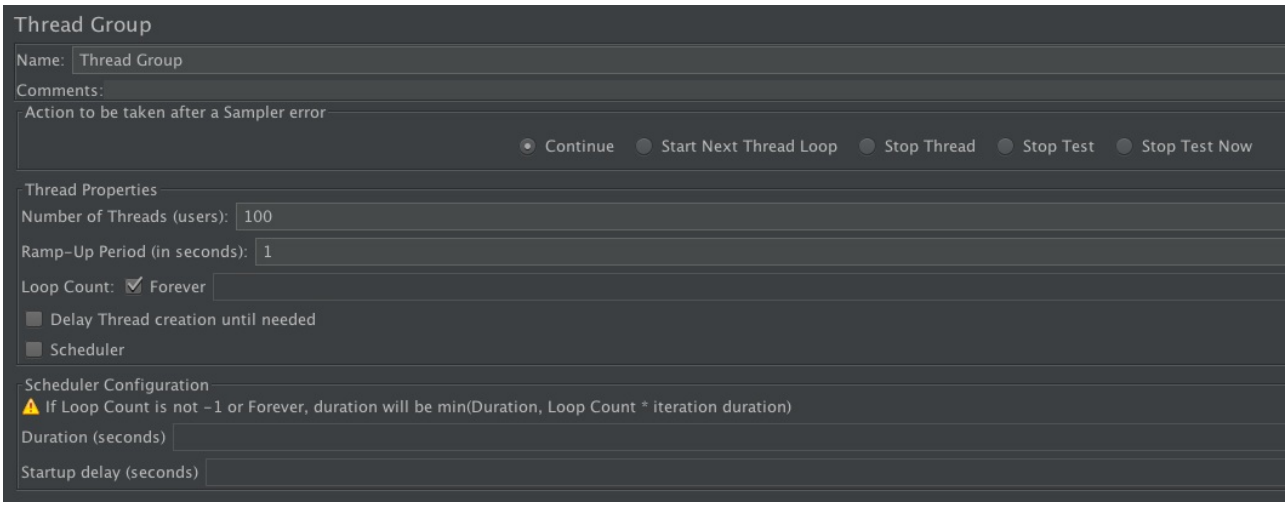
```
        return greeting;
    }
}
```

从上面的代码我们看到，程序会读取URL传过来的seconds参数，先休眠相应的秒数，再返回请求。这样做的目的是，客户端压测工具能够控制服务端的延迟。

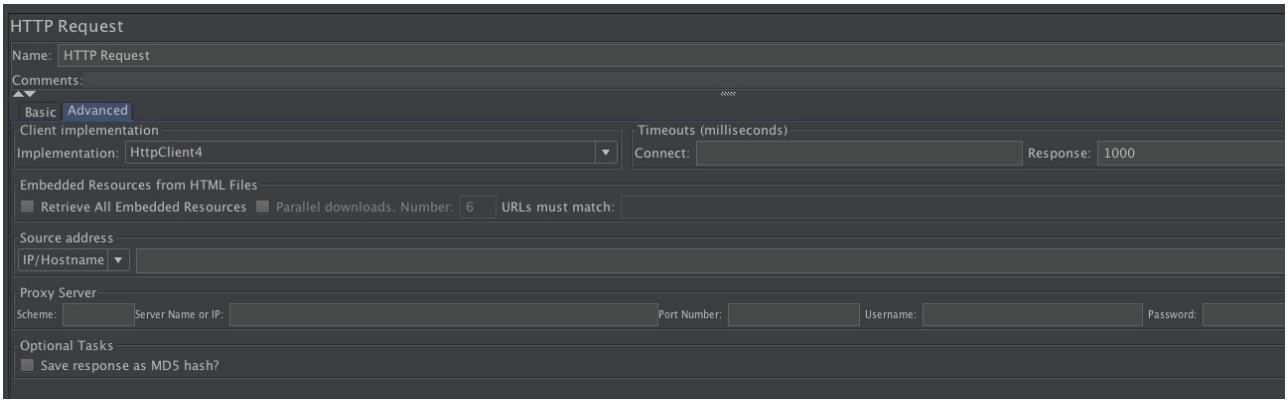
为了方便观察Tomcat的线程数跟延迟之间的关系，还需要加大Tomcat的最大线程数，我们可以在`application.properties`文件中加入这样一行：

```
server.tomcat.max-threads=1000server.tomcat.max-threads=1000
```

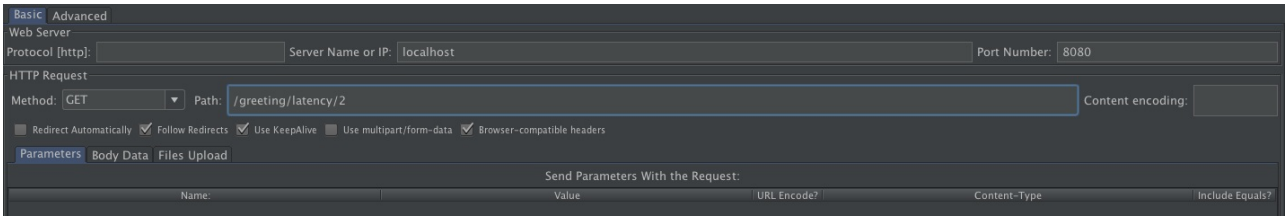
2.启动JMeter开始压测，这里我们将压测的线程数设置为100：



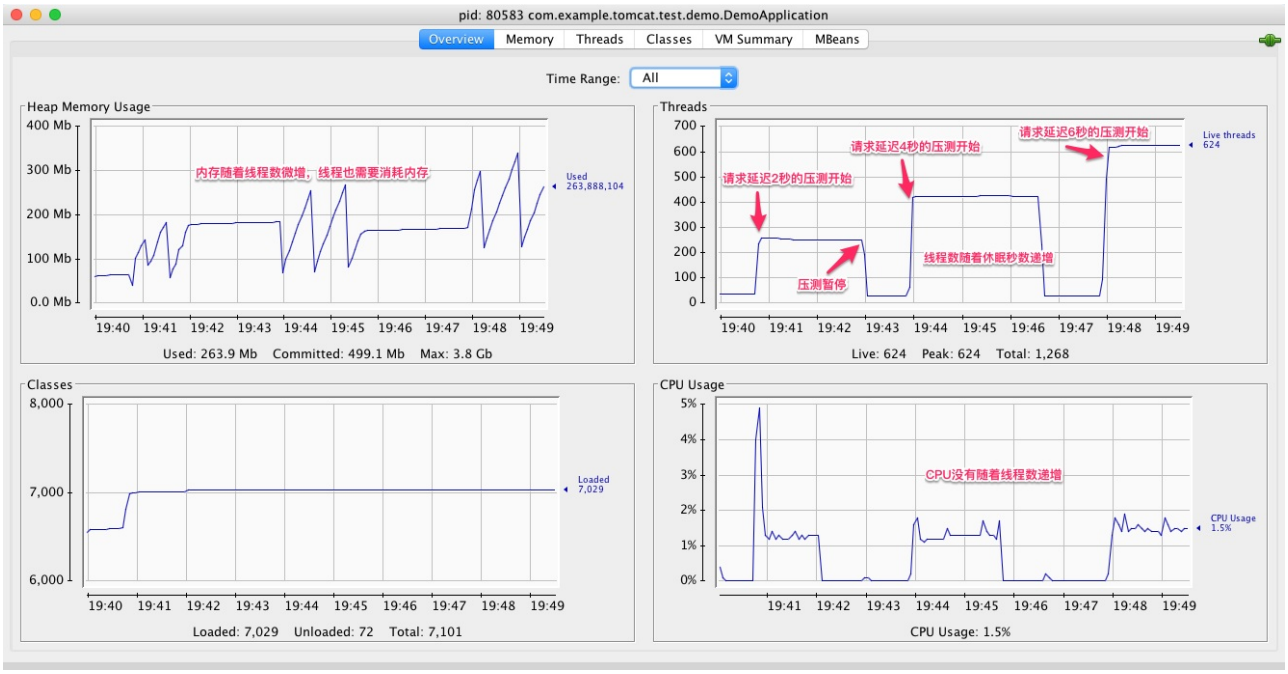
请你注意的是，我们还需要将客户端的Timeout设置为1000毫秒，这是因为JMeter的测试线程在收到响应之前，不会发出下一次请求，这就意味我们没法按照固定的吞吐量向服务端加压。而加了Timeout以后，JMeter会有固定的吞吐量向Tomcat发送请求。



3.开启测试，这里分三个阶段，第一个阶段将服务端休眠时间设为2秒，然后暂停一段时间。第二和第三阶段分别将休眠时间设置成4秒和6秒。



4.最后我们通过JConsole来观察结果：



下面我们从线程数、内存和CPU这三个指标来分析Tomcat的性能问题。

- 首先看线程数，在第一阶段时间之前，线程数大概是40，第一阶段压测开始后，线程数增长到250。为什么是250呢？这是因为JMeter每秒会发出100个请求，每一个请求休眠2秒，因此Tomcat需要200个工作线程来干活；此外Tomcat还有一些其他线程用来处理网络通信和后台任务，所以总数是250左右。第一阶段压测暂停后，线程数又下降到40，这是因为线程池会回收空闲线程。第二阶段测试开始后，线程数涨到了420，这是因为每个请求休眠了4秒；同理，我们看到第三阶段测试的线程数是620。
- 我们再来看CPU，在三个阶段的测试中，CPU的峰值始终比较稳定，这是因为JMeter控制了总体的吞吐量，因为服务端用来处理这些请求所需要消耗的CPU基本也是一样的。
- 各测试阶段的内存使用量略有增加，这是因为线程数增加了，创建线程也需要消耗内存。

从上面的测试结果我们可以得出一个结论：对于一个Web应用来说，下游服务的延迟越大，Tomcat所需要的线程数越多，但是CPU保持稳定。所以如果你在实际工作碰到线程数飙升但是CPU没有增加的情况，这个时候你需要怀疑，你的Web应用所依赖的下游服务是不是出了问题，响应时间是否变长了。

本期精华

今天我们学习了Tomcat中的关键的性能指标以及如何监控这些指标：主要有**吞吐量、响应时间、错误数、线程池、CPU以及JVM内存。**

在实际工作中，我们需要通过观察这些指标来诊断系统遇到的性能问题，找到性能瓶颈。如果我们监控到CPU上升，这时我们可以看看吞吐量是不是也上升了，如果是那说明正常；如果不是的话，可以看看GC的活动，如果GC活动频繁，并且内存居高不下，基本可以断定是内存泄漏。

课后思考

请问工作中你如何监控Web应用的健康状态？遇到性能问题的时候是如何做问题定位的呢？

不知道今天的内容你消化得如何？如果还有疑问，请大胆的在留言区提问，也欢迎你把你的课后思考和心得记录下来，与我和其他同学一起讨论。如果你觉得今天有所收获，欢迎你把它分享给你的朋友。



精选留言：

- nightmare 2019-08-01 01:11:38
  先查看日志，那些方法耗时较大，用阿里爸爸开源arthas监控有问题的方法，排查问题 [2赞]

- 许童童 2019-08-01 15:38:30
  使用prometheus + grafana 监控各种指标，每天上班前看一下昨天的情况，设置好阈值，如果达到就报警。 [1赞]

- 陆离 2019-08-01 13:49:41
  老师，不是很明白线程sleep时间越长，为什么tomcat启动的线程就越多

  作者回复2019-08-01 20:13:12
  这是Tomcat需要从线程池拿出一个工作线程来处理请求，请求处理（休眠）的时间越长，这些线程被阻塞，休眠时间越长，被阻塞的线程越多，这些线程无法被线程池回收，Tomcat线程池不得不创建更多的线程来处理新的请求。

- echo__陈 2019-08-01 12:59:19
  生产环境网络隔离……没办法适用jmx

- -W.LI- 2019-08-01 09:21:05
  李老师好。
  第二三阶段，有很多TIME_WAIT状态的线程。可是CPU使用率并没有增加很多。老师说CPU使用率和吞吐量有关，吞吐量由JMETER控制一直没变。频繁线程切换会消耗很多CPU支援，如果只是阻塞，切换不频繁。对CPU使用率还是比较小的是么?

- a、 2019-08-01 09:16:19
  监控系统会每隔一段时间，ping下我们系统，我们系统会pong回监控系统，并带上ip地址，jvm当前使用率，cpu使用率等信息，如果超过一定数值，监控系统就会发出预警信息，我们就需要去生产管理通过日志和命令查看，到底出了什么问题。